AEROSP 740: Assignment 3

Note 1: Completed solutions should be uploaded into Gradescope in a single pdf file before 11:59 p.m. on the day due. **Put boxes around final answers (except for problems involving derivations/proofs, simulation figures and MATLAB codes). 1-2 Points could be deducted if not following the instruction**. Carefully explain and justify your solutions. Plots should be clearly labeled. Include print out of all the codes. Homework should be neat in appearance. You can develop your own code from scratch or follow coding hints given.

Note 2: When submitting the assignment, please follow the steps in the Gradescope and assign corresponding pages to each problem number. **5 points** deduction will be taken if not following this procedure.

Note 3: This assignment pdf together with other complementary documents are uploaded into Canvas: Files > Homeworks and Solutions > Homework 3.

Note 4: For other homework related policies, please consult the syllabus.

1. (10 points) Suppose an output constraint, defined as

$$y_{min} \leq y = Cx + Du \leq y_{max},$$

where y could be a vector and C and D are given matrices, is added to the MPC problem formulation considered in Module 4. Derive a modified QP problem that needs to be solved to determine MPC action, i.e., what modifications to H, G, W, T, q will the introduction of such an output constraint induce? Assume the constraint is imposed in the MPC formulation as $y_{min} \leq y_k \leq y_{max}$, $k = 0, \cdots, N-1$.

Solution: Let the decision vector be the same as in the previous formulations of MPC problem.

$$U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix}.$$

In this case, we can impose constraints on

$$Y = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{bmatrix}$$

as

$$Y_{min} \leq Y \leq Y_{max},$$

and where

$$Y_{max} = \begin{bmatrix} y_{max} \\ \vdots \\ y_{max} \end{bmatrix}, \quad Y_{min} = \begin{bmatrix} y_{min} \\ \vdots \\ y_{min} \end{bmatrix}.$$

Note that

$$\begin{gathered} y_0 = Cx_0 + Du_0 \\ y_1 = CAx_0 + CBu_0 + Du_1 \\ \vdots \\ y_{N-1} = CA^{N-1}x_0 + CA^{N-2}Bu_0 + \cdots + CBu_{N-2} + Du_{N-1}. \end{gathered}$$

Hence we can write

$$Y = \Phi U + \Gamma x_0,$$

where

$$\Phi = \begin{bmatrix} D & 0 & \cdots & 0 & 0 \\ CB & D & 0 & \cdots & 0 \\ \vdots & & & & \\ CA^{N-2}B & \cdots & CB & D & \end{bmatrix}, \quad \Gamma = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{N-1} \end{bmatrix}.$$

Thus the affine constraints on states, inputs and outputs have the following form,

$$GU \leq W + Tx_0,$$

where

$$G = \begin{bmatrix} S \\ -S \\ I \\ -I \\ \Phi \\ -\Phi \end{bmatrix}, \quad T = \begin{bmatrix} -M \\ M \\ 0 \\ 0 \\ -\Gamma \\ \Gamma \end{bmatrix}, \quad W = \begin{bmatrix} X_{max} \\ -X_{min} \\ U_{max} \\ -U_{min} \\ Y_{max} \\ -Y_{min} \end{bmatrix},$$

where S, M are as previously defined in the lecture. Since the decision vector remained the same as previously, H and q do not change when output constraints are introduced. However, G, W and T do change.

---

2. The spacecraft attitude dynamics are represented by nonlinear ODEs

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \frac{1}{\cos(\theta)} \begin{bmatrix} \cos(\theta) & \sin(\phi)\sin(\theta) & \cos(\phi)\sin(\theta) \\ 0 & \cos(\phi)\cos(\theta) & -\sin(\phi)\cos(\theta) \\ 0 & \sin(\phi) & \cos(\phi) \end{bmatrix} \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix},$$

$$\begin{bmatrix} \dot{\omega}_1 \\ \dot{\omega}_2 \\ \dot{\omega}_3 \end{bmatrix} = \begin{bmatrix} ((J_2 - J_3)\omega_2\omega_3)/J_1 + M_1/J_1 \\ ((J_3 - J_1)\omega_3\omega_1)/J_2 + M_2/J_2 \\ ((J_1 - J_2)\omega_1\omega_2)/J_3 + M_3/J_3 \end{bmatrix},$$

where ϕ, θ, ψ are spacecraft roll, pitch and yaw angles, respectively; ω_1, ω_2, ω_3 are components of spacecraft angular velocity vector in the body fixed frame; $J_1 = 120$, $J_2 = 100$, $J_3 = 80$

are spacecraft principal moments of inertia; and $u = [M_1,\ M_2,\ M_3]^{\mathrm{T}}$ is the vector of control moments. Let $x = [\phi,\ \theta,\ \psi,\ \omega_1,\ \omega_2,\ \omega_3]^{\mathrm{T}}$ denote the state vector. The control moments are limited and control constraints are given by

$$-0.1 \leq M_i \leq 0.1, \quad i = 1, 2, 3.$$

The objective is to design and simulate an LQ-MPC controller on the nonlinear model. The controller must detumble the spacecraft and achieve the desired orientation, specifically, the objective is to steer the spacecraft state to $x = 0$. The state and control weighting matrices are given by

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.01 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.01 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.01 \end{bmatrix}, \ R = \begin{bmatrix} 0.01 & 0 & 0 \\ 0 & 0.01 & 0 \\ 0 & 0 & 0.01 \end{bmatrix},$$

and the terminal weighting matrix should be chosen as the solution to the corresponding DARE.

(a) (2 points) Implement a function of the form

begin code

```
function [xdot] = scdynamics(t,x,u)
```

end code

that can be passed to `ode45.m` to simulate the spacecraft attitude trajectory. As your answer, give a print-out of the code of `scdynamics`.

Solution:

begin code

```
function [xdot] = scdynamics(t,x,u)

global J1 J2 J3
% principal moments of inertia of spacecraft

phi   = x(1); % roll
theta = x(2); % pitch
psi   = x(3); % yaw

om1   = x(4); % first component of the angular velocity
                          % in the body fixed frame
om2   = x(5); % second component of the angular velocity
                          % in the body fixed frame
om3   = x(6); % third component of the angular velocity in
                          % the body fixed frame


% Kinematic equations
adot = 1/cos(theta)*[cos(theta); sin(phi)*sin(theta),
cos(phi)*sin(theta);
0, cos(phi)*cos(theta), -sin(phi)*cos(theta);
```

```
0, sin(phi), cos(phi)]*[om1;om2;om3];

phidot   = adot(1); % time rate of change of roll
thetadot = adot(2); % time rate of change of pitch
psidot   = adot(3); % time rate of change of yaw

% External moments
M  = u(:);
M1 = M(1);
M2 = M(2);
M3 = M(3);

% Dynamic equations
om1dot = ((J2-J3)*om2*om3)/J1 + M1/J1;
om2dot = ((J3-J1)*om3*om1)/J2 + M2/J2;
om3dot = ((J1-J2)*om1*om2)/J3 + M3/J3;

xdot = [phidot, thetadot, psidot, om1dot, om2dot, om3dot]';

return;

```

end code

(b) (2 points) Find the continuous-time linearized model at $x = 0$, $u = 0$ of the form,

$$\dot{x} = A_c x + B_c u,$$

and convert to discrete-time assuming the sampling period of $T_s = 2$ sec. Use `c2d.m`. As your answer, give the matrices A_d, B_d in the discrete-time model, $x_{k+1} = A_d x_k + B_d u_k$.

Solution:

$$A_d = \begin{bmatrix} 1 & 0 & 0 & 2 & 0 & 0 \\ 0 & 1 & 0 & 0 & 2 & 0 \\ 0 & 0 & 1 & 0 & 0 & 2 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad B_d = \begin{bmatrix} 0.0167 & 0 & 0 \\ 0 & 0.0200 & 0 \\ 0 & 0 & 0.0250 \\ 0.0167 & 0 & 0 \\ 0 & 0.0200 & 0 \\ 0 & 0 & 0.0250 \end{bmatrix}$$

(c) (8 points) Use the MPT3 toolbox to design the MPC controller. Develop closed-loop simulations to the initial condition, $x_0 = [-0.4,\ -0.8,\ 1.2,\ -0.02,\ -0.02,\ 0.02]^{\mathrm{T}}$. Your simulation should compute and hold the control constant over each time window of length T_s while simulating the underlying continuous-time nonlinear spacecraft dynamics, i.e., `scdynamics.m`, using `ode45.m`. For two horizons, $N = 2$ and $N = 20$ provide plots of the time histories of the orientation angles, angular velocity components, and control moments. Provide also plots of the time to compute the control action versus time. Show these responses for 200 sec. To determine the computation time, you can use the `tic` and `toc` commands in Matlab. For instance, suppose your controller is defined somewhere in your code as

begin code
```
ctrl = MPCController(model,N);
```
end code

You can use the following code snippet to compute the control action and time to compute the control action based on the state, x.

begin code
```
tic;
u  = ctrl.evaluate(x);
execTime = toc;
```
end code

Once you compute u, you can call `ode45.m` to perform the integration over the sampling period, $[t, t + T_s]$:

begin code
```
[T,X]   = ode45(@ (t,x) scdynamics(t,x,u), [t+h:h:t+Ts], x,options);
```
end code

where h can be chosen as $T_s/10$. You need to repeat this process of computing and applying the control action to obtain responses over the 200 sec time interval.

Solution:

begin code
```
clear all;
close all;
warning('off')

% +++ define spacecraft principal moments of inertia, kgm2
global J1 J2 J3
J1 = 120; J2 = 100; J3= 80;

% +++ Linearized model

Ac=[    0    0    0  1  0  0
        0    0    0  0  1  0
        0    0    0  0  0  1
        0    0    0  0  0  0
        0    0    0  0  0  0
        0    0    0  0  0  0];

Bc=[    0,      0,     0;
        0       0,     0;
        0,      0,     0;
        1/J1,   0,     0;
        0,      1/J2,   0;
        0,      0,      1/J3];

% +++  Convert the model to discrete-time
Ts     = 2.0; % Sampling period, sec
sysd   = c2d ( ss(Ac, [Bc],...
                [1,0,0,0,0,0;0,1,0,0,0,0;0,0,1,0,0,0],...
```

```
                zeros(3,3)), Ts);

[Ad, Bd, Cd, Dd, Ts] = ssdata(sysd); % Discrete matrices

% +++  Convert the model to discrete-time
Ts     = 2.0; % Sampling period, sec
sysd   = c2d ( ss(Ac, [Bc],...
[1,0,0,0,0,0;0,1,0,0,0,0;0,0,1,0,0,0], zeros(3,3)), Ts);

[Ad, Bd, Cd, Dd, Ts] = ssdata(sysd); % Discrete matrices

model = LTISystem('A', Ad , 'B', Bd, 'C', Cd, 'D', Dd, 'Ts', Ts);

% +++ Define constraints on outputs and inputs
model.u.max =  [ 0.1, 0.1, 0.1 ]';
model.u.min = -model.u.max;


% +++ Define Q and R weighting matrices of running cost
Q = diag([1,1,1,0.01,0.01,0.01]);
R = diag([1,1,1])*0.01;
model.x.penalty = QuadFunction(Q);
model.u.penalty = QuadFunction(R);

% +++ Compute terminal weighting matrix as a solution of
%     LQR problem
[KInf, PInf, E] = dlqr(Ad, Bd, Q, R);

model.x.with('terminalPenalty');
model.x.terminalPenalty = QuadFunction(PInf);

% Horizon
N = 2;
N = 20;


ctrl = MPCController(model,N);

% +++ run closed-loop simulations with nonlinear model
options = odeset('RelTol', 1e-8);
x0 = [-0.4;-0.8;1.2;-0.02;-0.02;0.02];
% initial condition = three Euler angles and
% three angular velocity components

t1  = 200; % final time (sec)
t   = 0;    % initial time
x   = x0;
TXU = [];
```

```
h=Ts/10;
global u
u_old=[0;0;0];
while t<=t1,
        try,
                tic;
                u  = ctrl.evaluate(x);
                execTime=toc;
        catch,
                execTime=NaN;
                disp(['ctrl.evaluate did not succeed,...
                      t=', num2str(t)]);
        end;
        if isempty(u) | isnan(u),
                disp(['ctrl.evaluate did not succeed,...
                 t=', num2str(t)]);
                 u = u_old;
        end;

        [T,X]   = ode45(@ (t,x) scdynamics(t,x,u),...
                        [t+h:h:t+Ts], x, options);

        TXU=[TXU;T(:),X,u(1)*ones(size(T)),...
        u(2)*ones(size(T)),u(3)*ones(size(T)),...
        execTime*ones(size(T))];

        x = X(end,:)';
        t = t + Ts
        u_old = u;
end;

figure;plot(TXU(:,1),TXU(:,2:4));
xlabel('t [sec]','fontsize',16);
legend('\phi','\theta','\psi');
set(gca,'fontsize',16)

figure;plot(TXU(:,1),TXU(:,5:7));
xlabel('t [sec]','fontsize',16);
legend('\omega_1','\omega_2','\omega_3');
set(gca,'fontsize',16)

figure;plot(TXU(:,1),TXU(:,8:10));
xlabel('t [sec]','fontsize',16);
legend('M_1','M_2','M_3');
set(gca,'fontsize',16)

figure;plot(TXU(:,1),TXU(:,11));
xlabel('t [sec]','fontsize',16);
```

```
125 legend('Execution Time');
126 set(gca,'fontsize',16)
```

end code

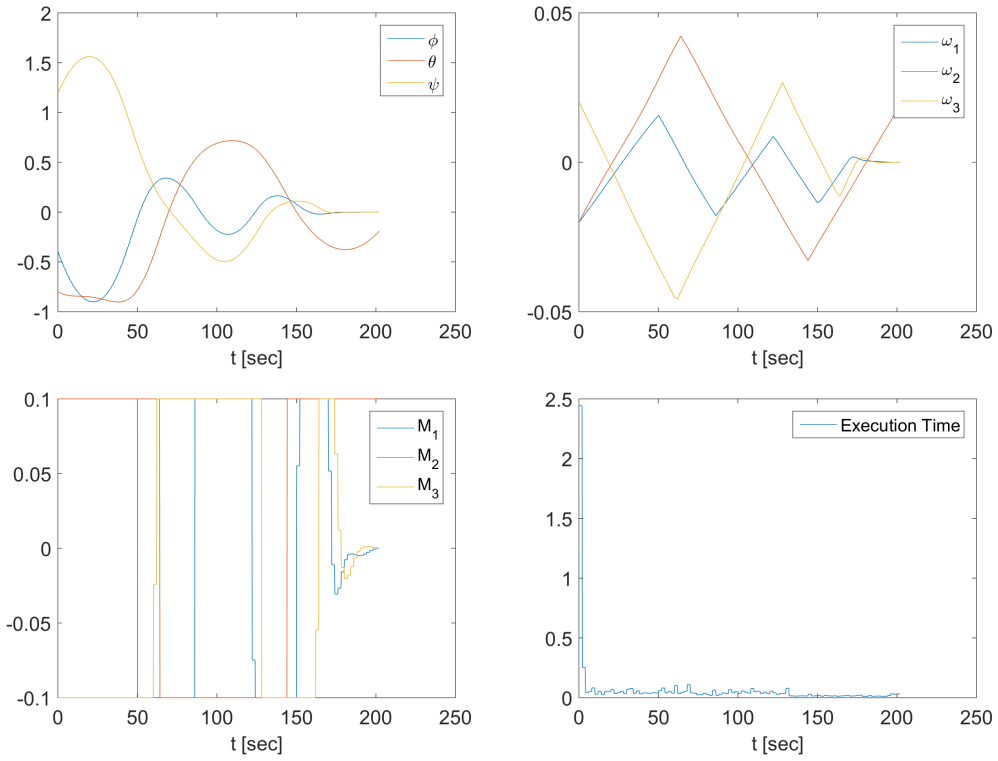

Figure 1: Time traces for the horizon $N = 2$, MPT3 toolbox.

---

3. We consider the control of a car lateral motion with active front steering. The linearized vehicle dynamics are given in continuous time by

$$\dot{x} = A_c x + B_c u,$$

where the components of the state vector $x \in \mathbb{R}^2$ are the lateral velocity of vehicle CG (x_1) and yaw rate (x_2), while the control, u, is the steering angle. The matrices A_c and B_c are given by

$$A_c = \begin{bmatrix} 2(C_f + C_r)/(mv_x) & 2(C_f l_f - C_r l_r)/(mv_x) - v_x \\ 2(l_f C_f - l_r C_r)/(I_z v_x) & 2(C_f l_f^2 + C_r l_r^2)/(I_z v_x) \end{bmatrix}, \quad B_c = \begin{bmatrix} -2C_f/m \\ -2l_f C_f/I_z \end{bmatrix}$$

where vehicle mass is $m = 1891$, forward velocity is $v_x = 20$, front tire cornering stiffness is $C_f = -18 \times 10^3$, rear tire cornering stiffness is $C_r = -28 \times 10^3$, $l_f = 1.5$, $l_r = 1.55$, and vehicle moment of inertia is $I_z = 3200$. The front slip angle is given by

$$\alpha_f = \frac{x_1}{v_x} + \frac{x_2 l_f}{v_x} - u,$$

and the rear slip angle is given by

$$\alpha_r = \frac{x_1}{v_x} - \frac{x_2 l_r}{v_x}.$$

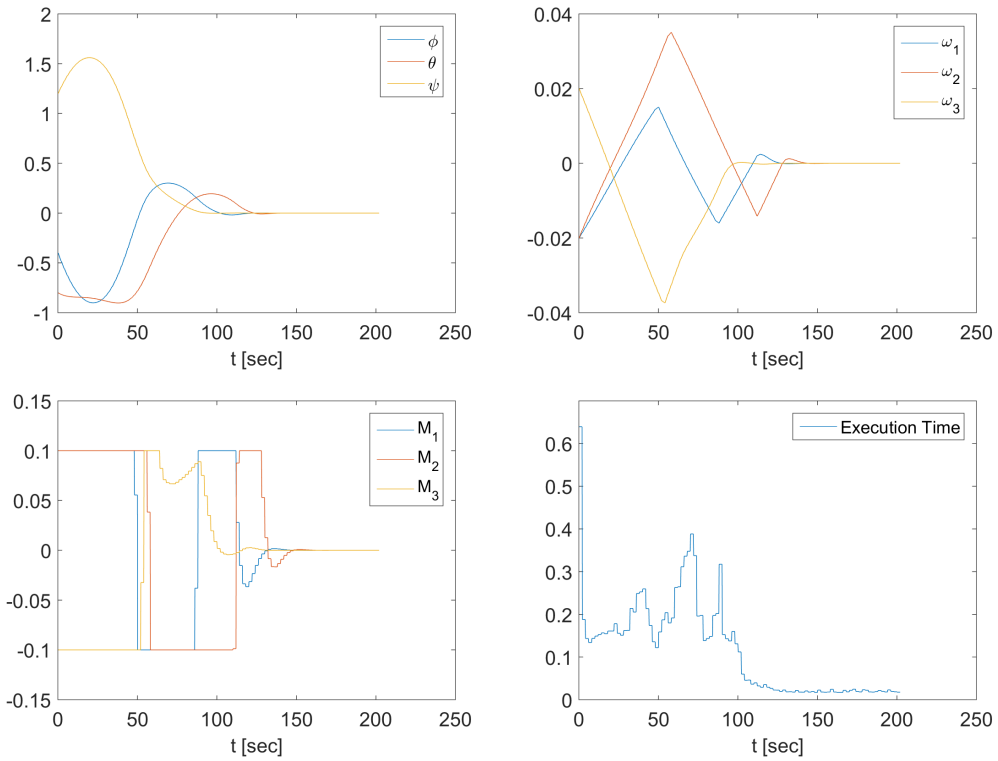

Figure 2: Time traces for the horizon $N = 20$, MPT3 toolbox.

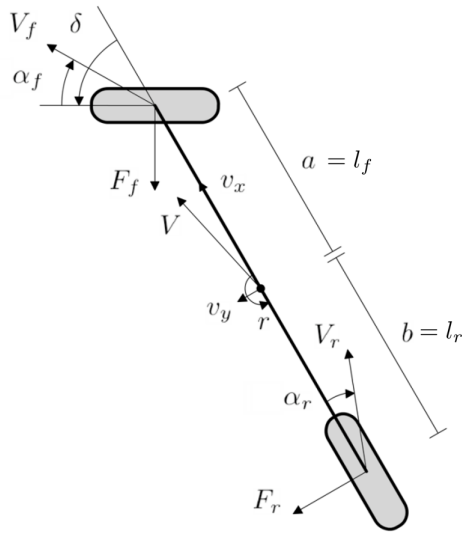

Figure 3: Vehicle schematics.

(a) (4 points) Assuming the sampling period of $T_s = 20 \times 10^{-3}$ sec, convert the linearized model to discrete-time. Use the `c2d.m` command. Give the discrete-time A_d and B_d matrices. Is A_d a Schur matrix? Is (A_d, B_d) controllable?

Solution:

$$A_d = \begin{pmatrix} 0.9507 & -0.3609 \\ 0.0097 & 0.9330 \end{pmatrix}, B_d = \begin{pmatrix} 0.3093 \\ 0.3281 \end{pmatrix}.$$

A_d is a Schur matrix as all eigenvalues are within the unit disk of the complex plane. (A_d, B_d) is controllable. This is verified by checking the rank of the controllability matrix, which is 2.

(b) (8 points) Consider the control objective of achieving offset-free tracking of the yaw rate command, r, i.e., we wish to achieve, $x_2 \to r$. Define the augmented state, $x_a = [x,\ u,\ r]^{\mathrm{T}}$ and the augmented model,

$$x_{a,k+1} = \begin{bmatrix} A_d & B_d & 0_{n_x \times 1} \\ 0_{1\times n_x} & 1 & 0 \\ 0_{1\times n_x} & 0 & 1 \end{bmatrix} x_{a,k} + \begin{bmatrix} B_d \\ 1 \\ 0 \end{bmatrix} \Delta u_k,$$

where $\Delta u_k = u_k - u_{k-1},\ n_x = 2$. Let $E_x = [0,\ 1,\ 0,\ -1]$ so that the tracking error can be expressed as $e = x_2 - r = E_x x_a$. Consider the constraints,

$$-3 \le x_1 \le 3,\ -1 \le x_2 \le 1,\ -0.15 \le u \le 0.15,$$

$$-0.1 \le \alpha_f \le 0.1,\ -0.07 \le \alpha_r \le 0.07,$$

and

$$-0.06 \le \Delta_u \le 0.06.$$

Use the MPT3 toolbox for the augmented system to generate the MPC Controller for the horizon $N = 10$ steps. For the weights, use

$$Q = E_x^{\mathrm{T}} Q_y E_x,\ Q_y = 3,\ R = 0.1,$$

and assume zero terminal penalty weight. Simulate the response to the zero initial condition for the horizon $N = 10$ steps. Assume that the yaw rate command, r, changes in steps between 0.2 and -0.2 staying at each level for 1 sec. Give plots of the time histories of yaw rate, side slip angle ($= x_1/v_x$ for small angles), steering angle, front slip angle and rear slip angle. Are the constraints satisfied?

Solution: begin code

```
clear all
close all

% vehicle parameters
m  = 1891 ;
vx = 20  ;
Cf = -18e3;
Cr = -28e3;
lf = 1.5 ;
lr = 1.55 ;
Iz = 3200;
r  = 0.2 ;
```

```
13
14  % modeling
15  Ts = 20e-3; %sampling period
16
17  Ac = [
18  2*(Cf+Cr)/(m*vx), 2*(Cf*lf-Cr*lr)/(m*vx)-vx; %beta= lateral velocity
19  2*(lf*Cf-lr*Cr)/(Iz*vx), 2*(Cf*lf^2+Cr*lr^2)/(Iz*vx)]; % yaw rate
20
21  Bc = [
22  -2*Cf/m
23  -2*lf*Cf/Iz]; % input = steering angle
24
25  Cc = [0 1 ; % yaw rate
26      1 0];   % lateral velocity
27
28  Dc = [0;0];
29
30  [Ad Bd Cd Dd] = ssdata(c2d(ss(Ac,Bc,Cc,Dc),Ts)) ;
31
32  % Augmented model for tracking with states (x,u,r) and control du
33  Ax = [Ad          Bd    0*Ad(:,1)
34      0*Ad(1,:)    1    0
35      0*Ad(1,:)    0    1     ] ; % states are x, u, r
36
37  Bx = [Bd;1;0] ;                  % delta u  = new control
38
39  Ex = [Cd(1,:) 0 -1] ;            % error = position - set-point
40
41  Cx=[ eye(3) zeros(3,1); % output vector = [x; u; alpha_f; alpha_r]
42      1/vx   lf/vx -1 0  ; % front slip angle
43      1/vx  -lr/vx  0 0  ; % rear slip angle
44          ] ;
45
46  Dx = [0;0;0;0;0] ;
47
48
49  model = LTISystem('A', Ax , 'B', Bx, 'C', Cx, 'D', Dx,...
50                                          'Ts', Ts);
51
52  % Control constraints (note on du)
53  model.u.min = -0.06;
54  model.u.max = -model.u.min;
55
56  % State constraints
57  model.y.max = [3, 1.0, 0.15, 0.1, 0.07]; % x, u, alpha_f, alpha_r
58  model.y.min = -model.y.max;
59
60  % Cost function
```

```
Qy = 3 ;
R  = 0.1;
model.u.penalty = QuadFunction(diag([R]));
model.x.penalty = QuadFunction(Ex'*Qy*Ex);

% Horizon
N=10;

% Generate MPC Controller
ctrl = MPCController(model,N)

% Simulate closed loop response
x0 = [0;0];
u  = 0;
r0 = 0.2;
Nsim = floor(2/Ts);
x = x0;
data.T=zeros(Nsim);
data.X=zeros(length(x0),Nsim);
data.U=zeros(size(Bd,2),Nsim);
data.R=zeros(1,Nsim);
data.Z=zeros(4,Nsim);

u2x = inv(eye(size(Ad))-Ad)*Bd;
u2y = Cd*u2x;

for i=1:Nsim,
        if i<5,
                xs = 0; us = 0;
                r =0;
        else,
                us = r/u2y(1);
                xs = u2x*us;
                r = r0*sign(sin(3*i*Ts));
        end;
        w = 0 ;
        try,
                [du,feasible,openloop] = ctrl.evaluate([x(:);u;r]);
        catch,
                du = 0;
        end
        u = u + du + w;
        data.T(i) = (i-1)*Ts;
        data.X(:,i)=x;
        data.U(:,i)=u;
        data.R(:,i)=r;
        data.Y(:,i)=(Ex(1,1:2)*x)';
        data.Z(:,i)=Cx([1,2,4,5],:)*[x',u,r]';
```

```
        x = Ad*x + Bd*(u+w);
        end;

        figure(24)
        subplot(311)
        plot(data.T,data.Y(1,:),'b',data.T,data.R);
        title('Yaw rate')
        subplot(312)
        plot(data.T,data.X(1,:)/vx,'b')
        title('Side slip angle')
        subplot(313)
        plot(data.T,data.U(1,:))
        title('Steering angle')
        xlabel('t [sec]')

        figure(25)
        subplot(211)
        plot(data.T,data.Z(3,:),'b')
        title('Front slip angle')
        subplot(212)
        plot(data.T,data.Z(4,:),'b')
        title('Rear slip angle')
```

end code

If you zoom in, the constraints on steering angle are strictly enforced. There is, however, a slight violation of the front slip angle constraints.

4. Consider the system represented by

$$x_{k+1} = Ax_k + Bu_k + w_k,\ y_k = Cx_k, \quad A = 0.5,\ B = 1.0,\ C = 1.0,$$

for which we want to solve a reference tracking problem, $y_k \to r$ as $k \to \infty$ assuming the disturbance, w_k, is a constant. We will use a *zero-placement* approach to solve this problem. Define

$$\Delta x_k = x_k - x_{k-1}, \quad \Delta u_k = u_k - u_{k-1}, \quad e_k = Cx_{k-1} - r,$$

and let

$$z_k = \begin{bmatrix} e_k \\ \Delta x_k \end{bmatrix}.$$

(a) (2 points) Assume $w_k = 0$ in MPC design stage, and write down the model in the form,

$$z_{k+1} = A_x z_k + B_x \Delta u_k.$$

Give A_x and B_x as your answer.

Solution:

$$A_x = \begin{bmatrix} 1 & C \\ 0 & 0.5 \end{bmatrix} \quad B_x = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

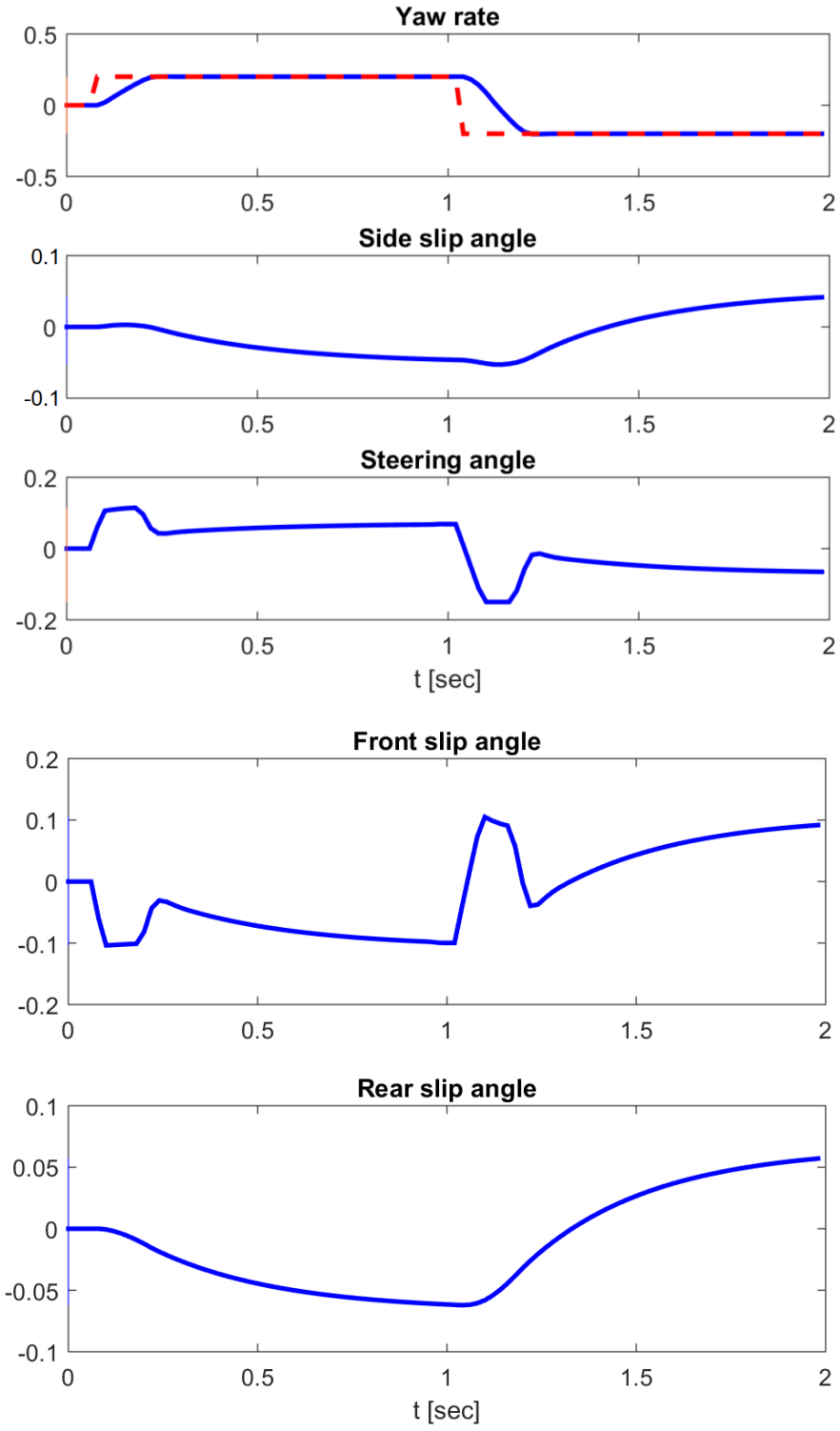

Figure 4: Time traces for the vehicle steering problem using MPT3.

(b) (2 points) Suppose that we want the tracking error to decay according to a prescribed

reference model, $e_{k+1} = \lambda e_k$, where $1 > \lambda > 0$ is a tunable parameter. Let a *sliding mode variable* be defined as

$$s_k = e_{k+1} - \lambda e_k.$$

Express s_k as $s_k = C_x z_k$ and give C_x as your answer.

Solution:

$$C_x = \begin{bmatrix} (1-\lambda) & 1 \end{bmatrix}$$

(c) (2 points) Note that we want to keep s_k closer to zero as then the error dynamics follow our reference model. Set the prediction horizon of your problem to $N = 2$, and set $R = 1$. Consider the cost function,

$$J = \sum_{k=0}^{N-1} s_k^\mathsf{T} s_k + \Delta u_k^\mathsf{T} R \Delta u_k,$$

which seeks to minimize s_k and induce the desired response dynamics. Convert this cost function to the form,

$$J = \sum_{k=0}^{N-1} z_k^\mathsf{T} Q_x z_k + \Delta u_k^\mathsf{T} R_x \Delta u_k,$$

and give Q_x and R_x as your answers.

Solution:

$$Q_x = \begin{bmatrix} (1-\lambda)^2 & (1-\lambda) \\ (1-\lambda) & 1 \end{bmatrix}, \quad R_x = 1.$$

(d) (10 points) Implement closed-loop system simulations using the MPT3 toolbox. For twenty simulation time steps, report the responses of x_k and r (one plot) and u_k (second plot) to (i) $r = 1$ for $\lambda = 0.1$ and $\lambda = 0.7$ when $w_k = 0$, $x_0 = 0$; (ii) $r = 1$ for $\lambda = 0.1$ and $\lambda = 0.7$ when $w_k = 1.0$, $x_0 = 0$;

Solution:

begin code

```
close all; clear all;
A = 0.5; B = 1.0;
C = 1.0; D = 0;
R = 1;

% Extended system and weights
Ax  = [1, C; 0, A];
Bx  = [0; B];
lam = 0.1;
Cx = [(1-lam),C];
Dx = 0;
Qx = Cx'*Cx;
Rx = R;

N  = 2; % Prediction horizon
```

```
Ts = 1.0; % Sampling period

model = LTISystem('A',Ax,'B',Bx,'C',Cx,'D',Dx, 'Ts', Ts);

% Control constraints
model.u.min= -Inf;
model.u.max= -model.u.min;

% State constraints
model.x.min=[-Inf, -Inf];
model.x.max=[Inf, Inf];

% Cost function
model.u.penalty = QuadFunction(Rx);
model.x.penalty = QuadFunction(Cx'*Cx);

% Generate MPC Controller
ctrl = MPCController(model,N)


% Simulate closed loop response
x0 = [0]; u  = 0;
r0 = 1; e  = C*x0 - r0;
dx = 0;
Nsim = 21;
x = x0;
data.X = zeros(length(x0),Nsim);
data.U = zeros(size(B,2),Nsim);
data.R = zeros(1,Nsim);
data.W = zeros(1,Nsim);
for i=1:Nsim,
r = 1.0; w = 0.0;
try,
[du,feasible,openloop] = ctrl.evaluate([e;dx(:)]);
catch,
du = 0;
end
if isnan(du), du=0; end;
u = u + du;
data.X(:,i) = x;
data.U(:,i) = u;
data.R(:,i) = r;
data.W(:,i) = w;
x_next = A*x + B*(u+w);
dx = x_next - x;
e  = C*x(:) - r;
x  = x_next;
end;
```

```
64
65          % Plot the responses
66          figure;
67          subplot(311)
68          plot(([1:Nsim]-1)*Ts, data.X(1,:),'bo-', ([1:Nsim]-1)*Ts,...
69          data.R, 'g--');
70          ylabel('x_1'); set(gca,'fontsize',16)
71          subplot(312)
72          plot(([1:Nsim]-1)*Ts, data.U(1,:),'o-')
73          ylabel('u'); set(gca,'fontsize',16)
74          subplot(313)
75          plot(([1:Nsim]-1)*Ts, data.W(1,:),'o-')
76          ylabel('w'); set(gca,'fontsize',16)
77          xlabel('k')
78
```

end code

Simulation results are shown in Figures 5-6 Note that the controller is noticeably faster when $\lambda = 0.1$. We also observe that the controller has the offset-free behavior as the output tracks the reference command $r = 1$ with or without the disturbance. Note that the disturbance causes an overshoot in the initial response and it is no longer first order as intended.

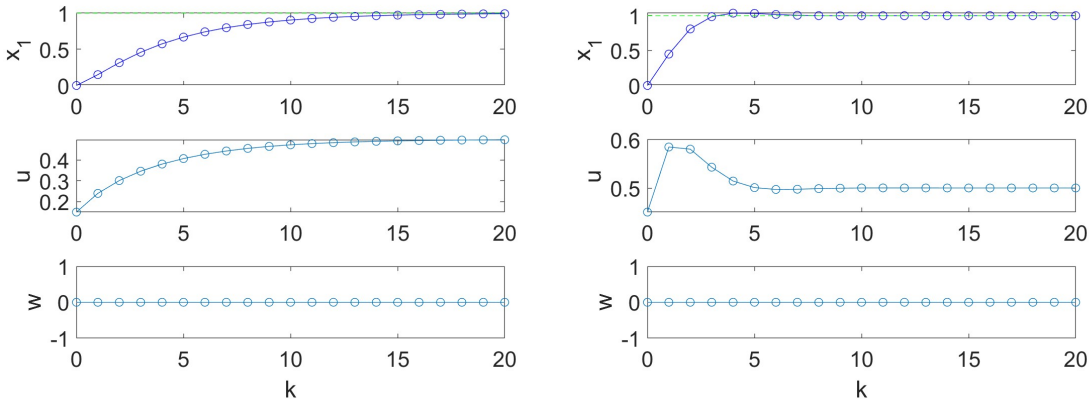

Figure 5: Simulations for $w = 0$ and $\lambda = 0.7$ (Left) and $\lambda = 0.1$ (Right).

---

5. The spacecraft relative motion dynamics (in orbital plane relative to a nominal orbital position on a circular orbit) without thrust/other external forces acting are described by the following differential equations

$$\ddot{x} = 3n^2x + 2n\dot{y}$$
$$\ddot{y} = -2n\dot{x},$$
$$\ddot{z} = -n^2z$$

where x, y, z are the spacecraft relative position coordinates in km, and $n = 0.00114$ is the mean motion in rad/sec.

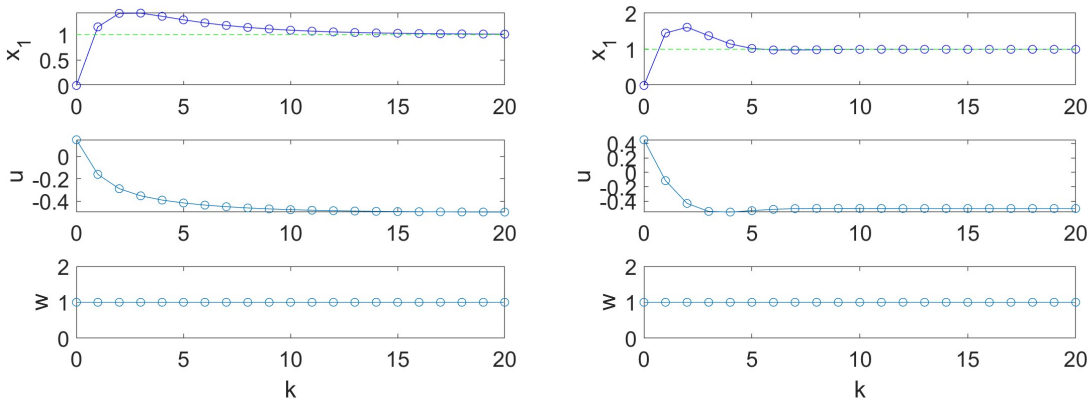

Figure 6: Simulations for $w = 1$ and $\lambda = 0.7$ (Left) and $\lambda = 0.1$ (Right).

(a) (2 points) Let the state vector be $X = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^{\mathrm{T}}$. Write out equations of motion in the state space form, $\dot{X} = A_c X$. Give A_c as your answer. Describe stability properties of the open-loop spacecraft relative motion dynamics. You can answer this question for the model with the numerical parameter values as given.

Solution:

$$A_c = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 3n^2 & 0 & 0 & 0 & 2n & 0 \\ 0 & 0 & 0 & -2n & 0 & 0 \\ 0 & 0 & -n^2 & 0 & 0 & 0 \end{bmatrix},$$

There is a repeated eigenvalue of A_c at 0 and the algebraic and geometric multiplicities of it are not the same. Consequently, the dynamics are open loop unstable.

(b) (2 points) Assume that the output measurements are

$$Y = \begin{bmatrix} x \\ y \\ z \end{bmatrix},$$

and you wish to write the output equation in the standard form, $Y = CX$. What is C matrix? Is the matrix pair (C, A) observable?

Solution:

begin code

```
1  C=[1,0,0,0,0,0;0,1,0,0,0,0;0,0,1,0,0,0]
```

end code

The pair (C, A) is observable as the rank of the observability matrix is equal to 6.

(c) (2 points) Assume that the output is sampled with the sampling period of $T_s = 30$ sec. Using the c2d command, obtain discrete-time equations of motion in the form $X_{k+1} = A_d X_k$, $Y_k = C_d X_k$. Give A_d and C_d as your answers.

Solution:

$$A_d = \begin{bmatrix} 1.0018 & 0 & 0 & 29.9942 & 1.0259 & 0 \\ -3.9999e-05 & 1 & 0 & -1.0259 & 29.9766 & 0 \\ 0 & 0 & 0.99942 & 0 & 0 & 29.9942 \\ 0.00011694 & 0 & 0 & 0.99942 & 0.068387 & 0 \\ -3.9998e-06 & 0 & 0 & -0.068387 & 0.99766 & 0 \\ 0 & 0 & -3.898e-05 & 0 & 0 & 0.99942 \end{bmatrix}$$

$$C_d = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

(d) (8 points) Implement a Moving Horizon Observer (MHO) assuming the following MHO parameters:

begin code

```
P_0   = diag([1,1,1,0.001,0.001,0.001])
R     = diag([0.01,0.01,0.01])
Q     = diag([0.001,0.001,0.001,0.1,0.1,0.1])
```

end code

Suppose the true initial state is $X_0 = [1,\ 1,\ -1,\ 0.002,\ -0.002,\ 0.004]^\mathsf{T}$, and the initial state estimate is $\hat{X}_0 = [0,\ 0,\ 0,\ 0,\ 0,\ 0]^\mathsf{T}$. Suppose that the output is measured with a measurement noise,

begin code

```
Y = C_d*X + 0.01*randn(3,1)
```

end code

Simulate for $k = 0, 1, 2, \cdots, 100$ and plot the true position states x_k, y_k, z_k and the estimated position states, $\hat{x}_k, \hat{y}_k, \hat{z}_k$ on one plot and the true velocity states $\dot{x}_k, \dot{y}_k, \dot{z}_k$ and the estimated velocity states, $\hat{\dot{x}}_k, \hat{\dot{y}}_k, \hat{\dot{z}}_k$ on the other plot.

Solution:

begin code

```

Traj = [];
P   = diag([1,1,1,0.001,0.001,0.001])

R    = diag([0.01,0.01,0.01])
Q    = diag([0.001,0.001,0.001,0.1,0.1,0.1])

Xd   = [1.0, 1, -1.0, 0.002, -0.002, 0.004]';
Xdhat= [0,0,0,0,0,0]';

for k = 0:1:100,
Traj.time(k+1) = k;
Traj.Xd(k+1,:) = Xd;
Traj.Xdhat(k+1,:) = Xdhat;

Yd = Cd*Xd+0.01*randn(3,1); % true output measurement
```

```
            Ydhat = Cd*Xdhat;           % estimated output measurement

            Pprior = Q+Ad*P*Ad';

            L = Pprior*Cd'*inv(Cd*Pprior*Cd'+R);
            P = Pprior - Pprior*Cd'*inv(Cd*Pprior*Cd'+R)*Cd*Pprior;
            Xd = Ad*Xd; % update model
            Xdhatprior = Ad*Xdhat;
            Xdhat  = Xdhatprior  + L*(Yd-Cd*Xdhatprior);
            % update the observer

            end;
            figure;plot(Traj.time,Traj.Xd(:,1:3),'b-',...
                        Traj.time,Traj.Xdhat(:,1:3 ),'k--')
            ylabel('x,y,z [km]'); xlabel('k');
            set(gca,'fontsize',16)
            figure;plot(Traj.time,Traj.Xd(:,4:6),'b-',...
                       Traj.time,Traj.Xdhat(:,4:6 ),['k--'])
            ylabel('$\dot{x}, \dot{y}, \dot{z}$ [km/sec]',....
            'fontsize',14, 'interpreter','latex');
            xlabel('k'); set(gca,'fontsize',16)
```

end code

6. The spacecraft relative motion dynamics (in orbital plane relative to a nominal orbital position on a circular orbit) are described by the following differential equations

$$\ddot{x} = 3n^2 x + 2n\dot{y} + \frac{u_x}{m}$$
$$\ddot{y} = -2n\dot{x} + \frac{u_y}{m},$$

where x and y are the spacecraft position coordinates in km, u_x and u_y are the components of the thrust force vector in kN, $n = 1.107 \times 10^{-3}$ is the mean motion in rad/sec, and $m = 100$ is the spacecraft mass in kg.

(a) (2 points) Let the state vector be $X = [x, y, \dot{x}, \dot{y}]^{\mathrm{T}}$ and the control vector be $u = [u_x, u_y]^{\mathrm{T}}$. Write out equations of motion in the state space form, $\dot{X} = A_c X + B_c u$. Give A_c and B_c matrices as your answer. Describe the stability properties of the open-loop spacecraft dynamics. Are these dynamics controllable? You can answer these questions for the model with the numerical parameter values as given.

Solution:

$$A_c = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 3n^2 & 0 & 0 & 2n \\ 0 & 0 & -2n & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1/m & 0 \\ 0 & 1/m \end{bmatrix},$$

There is a repeated eigenvalue of A_c at 0 and the algebraic and geometric multiplicies of it are not the same. Consequently, the dynamics are open loop unstable. The rank of the controllability matrix is 4 and hence open-loop dynamics are controllable.

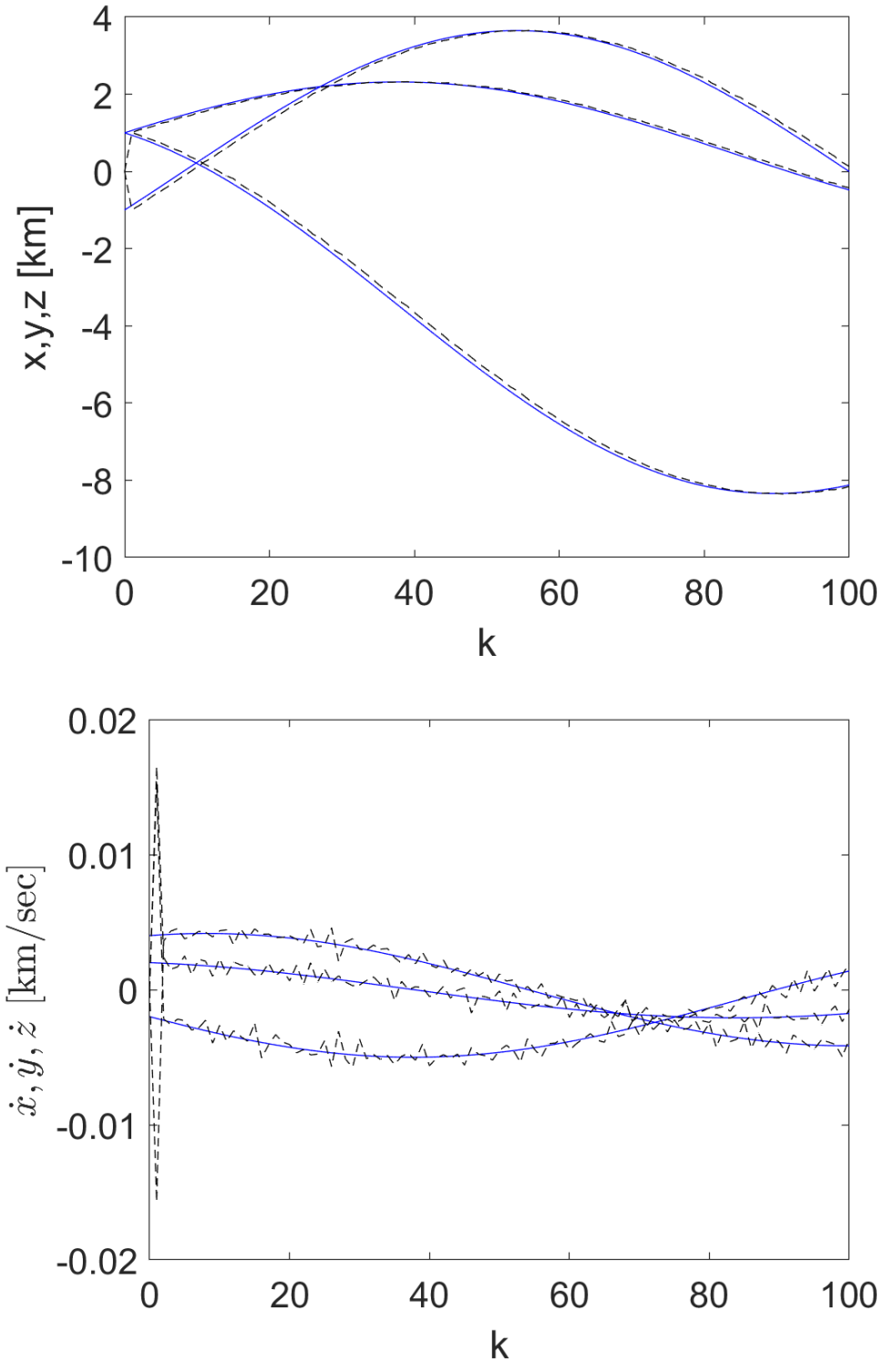

Figure 7: True and estimated with MHO states.

(b) (3 points) Assume the sampling period is $T_s = 20$ sec and convert the model to discrete-time

form using the `c2d.m` command. For the resulting discrete-time model, $X_{k+1} = A_d X_k + B_d u_k$, suppose the cost function that we wish to minimize has the form,

$$J_N = qX_N^T X_N + \sum_{k=0}^{N-1} \|u_k\|_1. \tag{1}$$

This 1-norm cost function reflects the actual fuel consumption of the spacecraft with independent thrusters along the x and y axes. The first term with the scalar weight $q > 0$ reflects our objective of driving the spacecraft to the origin (e.g., for a rendezvous or a docking mission). Let

$$U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix}$$

Using the state transition formula, you can express X_N in the form $X_N = PX_0 + RU$. Give expressions for P and R.

Solution:

$$P = A^N, \quad R = \begin{bmatrix} A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix}.$$

(c) (3 points) To handle the 1-norm cost, we introduce auxiliary variables, $\zeta_i \in \mathbb{R}^2$, $i = 0, \cdots, N-1$, and inequality constraints,

$$\begin{aligned} \zeta_0 \geq u_0, \quad & \zeta_0 \geq -u_0, \\ \zeta_1 \geq u_1, \quad & \zeta_1 \geq -u_1, \\ & \vdots \\ \zeta_{N-1} \geq u_{N-1}, \quad & \zeta_{N-1} \geq -u_{N-1}. \end{aligned}$$

We change the cost function to

$$\tilde{J}_N = qX_N^T X_N + \sum_{k=0}^{N-1} \mathbf{1}^{\mathrm{T}} \zeta_i,$$

where $\mathbf{1}^{\mathrm{T}} = \begin{bmatrix} 1 & 1 \end{bmatrix}$.
Let

$$Z = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \\ \zeta_0 \\ \zeta_1 \\ \vdots \\ \zeta_{N-1} \end{bmatrix},$$

be the "decision" vector. Give the expression for the cost $\tilde{J}_N$ as a quadratic function of Z, i.e., $\tilde{J}_N = \frac{1}{2} Z^{\mathrm{T}} \Gamma_1 Z + \gamma_2 Z +$ constant, where you determine expressions for Γ_1 and Γ_2. You can omit additive constant terms that do not depend on Z since they will not affect the result of the optimization.

Solution:

$$\tilde{J}_N = \frac{1}{2} Z^{\mathrm{T}} \begin{bmatrix} 2qR^{\mathrm{T}}R & 0_{2N\times 2N} \\ 0_{2N\times 2N} & 0_{2N\times 2N} \end{bmatrix} Z + \begin{bmatrix} 2qX_0^{\mathrm{T}}P^{\mathrm{T}}R & \mathbf{1}_{1\times 2N} \end{bmatrix} Z + \text{constant}$$

(d) (3 points) Suppose that minimizing $\tilde{J}_N$ yields a control sequence, $\{u_0^*, \cdots, u_{N-1}^*\}$. Justify mathematically that this sequence minimizes J_N.

Solution: Any feasible solution of the problem of minimizing J_N can be converted to a feasible solution of the problem of minimizing $\tilde{J}_N$ by defining $\zeta_i = |u_i|$ and both problems have the same cost, $J_N = \tilde{J}_N$. This implies that minimizing $\tilde{J}_N$ with respect to Z cannot produce worse results, in terms of the cost, than minimizing J_N with respect to U.

(e) (3 points) Let $N = 15$, $q = 10^4$ and $X_0 = [0,\ 10,\ 0,\ 0]^{\mathrm{T}}$. Using Matlab's quadratic programming solver, `quadprog.m`, solve the problem of minimizing $\tilde{J}_N$ subject to the above constraints. Extract from Z the corresponding control trajectory U and simulate the spacecraft motion based on your discrete-time model. Do not recompute the solution at subsequent time instants as in MPC, but just apply your computed input trajectory. Include a plot the spacecraft trajectory on y-x plane (note axes are switched). Include also a plot of the time histories of the control inputs versus time. Use Matlab's `stairs.m` command.

Solution: See Figures 8-9. The following code snippet was used to solve the problem

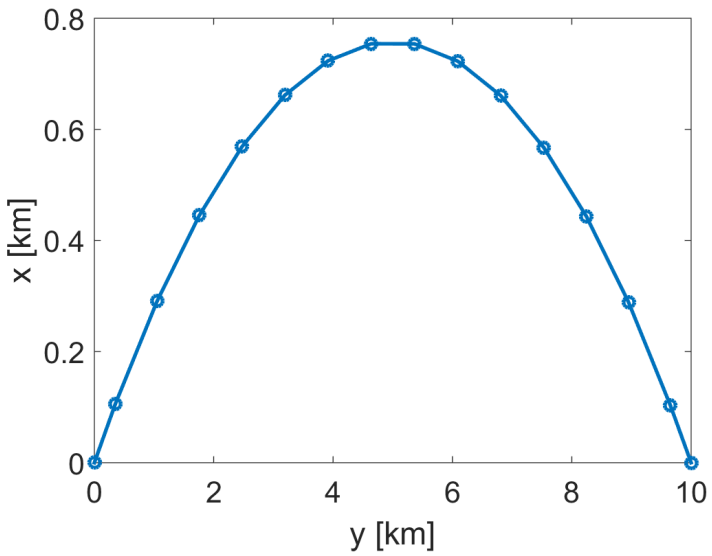

Figure 8: Spacecraft relative motion $y - x$ trajectory.

begin code

```
% Define the discrete-time horizon
N = 300/Ts;

% Initial condition
X0 = [0,10,0,0]';
```

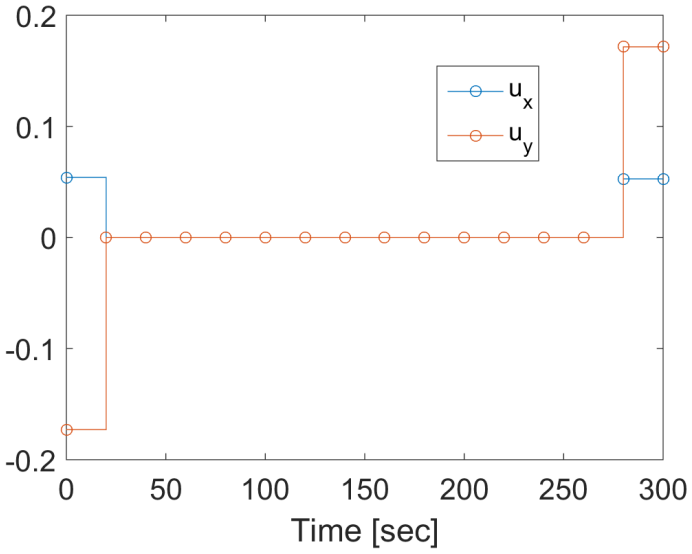

Figure 9: Thrust force components u_x and u_y.

```
6
7   % Weight
8   q = 10^4;
9   P = Ad^(N);
10  R = [];
11
12  for k=0:1:N-1,
13          R = [Ad^k*Bd,R];
14  end;
15
16  E = [eye(2*N,2*N),  -eye(2*N,2*N);
17      -eye(2*N,2*N), -eye(2*N,2*N)];
18
19  Z = quadprog(blkdiag(2*q*R'*R,zeros(2*N,2*N)),...
20               [2*q*X0'*P'*R, ones(1,2*N)],...
21               E, zeros(4*N,1));
22
23  U = [eye(2*N,2*N),zeros(2*N,2*N)]*Z; % control vector
24
25  u=zeros(2,N);
26  for k=0:1:N-1,
27          u(:,k+1)=[U(2*k+1:2*k+2)];
28  end;
29
30  % simulate
31  X  = X0;
32  Tr = [];
33  for k=0:1:N-1,
```

```
        Y = [1,0,0,0;0,1,0,0]*X;
        Tr = [Tr;k*Ts,Y(1),Y(2),u(1,k+1),u(2,k+1)];
        X = Ad*X + Bd*u(:,k+1);
end;
Y = [1,0,0,0;0,1,0,0]*X;
Tr=[Tr;N*Ts,Y(1),Y(2),u(1,end),u(2,end)];

figure(1);
myp = plot(Tr(:,3),Tr(:,2),'-o');
set(myp,'linewidth',2)
xlabel('y [km]')
ylabel('x [km]')
set(gca,'fontsize',16)

figure(2);stairs(Tr(:,1),Tr(:,4:5),'-o');
xlabel('Time [sec]');
legend('u_x','u_y')
set(gca,'fontsize',16)


```

end code

(f) (2 points) What is the "fuel cost" of the optimal maneuver, i.e., the value of $\sum_{k=0}^{N-1} \|u_k\|_1$?

Solution: We can use the following code snippet,

begin code

```
% Compute fuel cost
JN = 0;
for k=0:1:N-1,
        JN = JN + norm(u(:,k+1),1);
end;
JN
```

end code

The value of the "fuel cost" is

$$J_N = 0.4513$$

(g) (4 points) Suppose now we impose additional constraints,

$$\|u_k\|_\infty \leq 0.05,\ k = 0, \cdots, N-1.$$

Add these constraints to your quadratic program and repeat problems (6e) and (6f). Include the same type of plots and compute the "fuel cost." Explain the reasons the fuel cost has increased.

Solution: See Figures 10-11. The fuel cost is higher,

$$J_N = 0.5687,$$

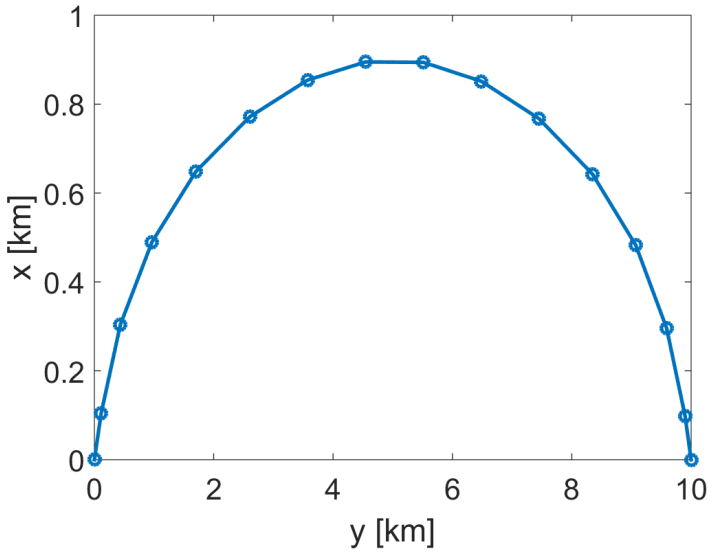

Figure 10: Spacecraft relative motion $y - x$ trajectory with extra constraints on thrust magnitude.

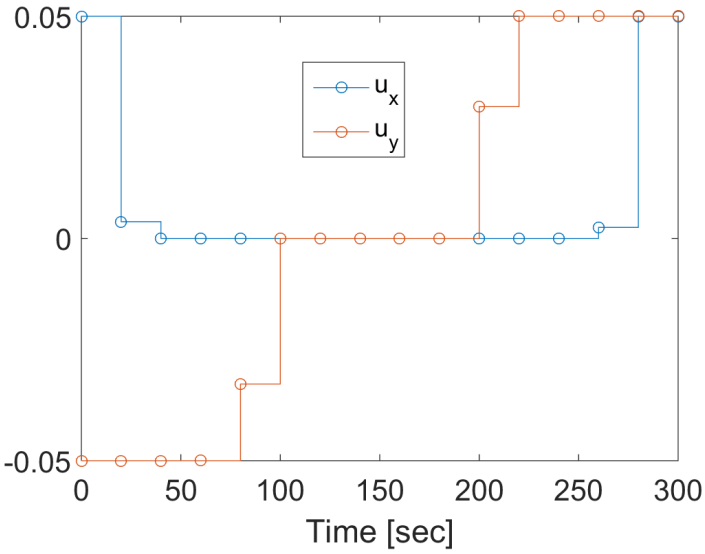

Figure 11: Thrust force components u_x and u_y with extra constraints on thrust magnitude.

as we have imposed additional constraints. The solution to a problem that has additional constraints cannot be better in terms of the cost than the solution without these constraints.